



GCT153 Estruturas de Dados II

Grafos: Rotas, Conectividade e Redes de Menor Custo

Juka Francis Pereira Gonçalves
Maria Julia Pedroso Pimenta

Sumário

1	Introdução	3
2	Representação do Grafo	3
3	Parte 1: Caminhos Mínimos com Dijkstra	4
3.1	Resultados obtidos	5
4	Parte 2: Árvore Geradora Mínima com Prim	6
4.1	Resultados obtidos	7
5	Parte 3: Árvore Geradora Mínima com Kruskal	8
5.1	Resultados obtidos	8
6	Comparação entre os Algoritmos	9
6.1	Prim vs. Kruskal	9
6.2	Dijkstra vs. Prim/Kruskal: rota vs. rede	10
7	Respostas às Perguntas de Análise	11
7.1	Parte 1: Dijkstra	11
7.2	Parte 2: Prim	12
7.3	Parte 3: Kruskal	13
8	Conclusão	13

1 Introdução

Este relatório apresenta o desenvolvimento, a implementação e a análise de três algoritmos clássicos de grafos: Dijkstra, Prim e Kruskal, aplicados a um cenário de planejamento de rede de distribuição entre cidades. O objetivo central do trabalho é compreender e demonstrar, na prática, a diferença fundamental entre dois tipos de problemas em grafos:

- **Menor caminho (rota):** encontrar o trajeto de menor custo entre uma cidade de origem e as demais, respondido pelo algoritmo de Dijkstra;
- **Menor custo de conexão total (rede):** encontrar a forma mais barata de conectar todas as cidades em uma única rede, respondido pelos algoritmos de Prim e Kruskal.

Todos os algoritmos foram implementados do zero, em Python, sem o uso de bibliotecas prontas de grafos (como NetworkX), utilizando apenas estruturas básicas da linguagem e o módulo `heapq` para a implementação de filas de prioridade, conforme permitido pelo enunciado.

2 Representação do Grafo

O grafo utilizado é não direcionado e ponderado, com $V = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ (9 vértices) e 15 arestas, conforme a tabela abaixo.

Tabela 1: Arestas do grafo de entrada

Origem	Destino	Peso
0	1	4
0	2	2
1	2	1
1	3	5
2	3	8
2	4	10
3	4	2
3	5	6
4	5	3
4	6	7
5	6	1
5	7	9
6	7	4
6	8	6
7	8	2

A representação escolhida foi a **lista de adjacência**, implementada por meio de uma lista de listas, em que cada posição i armazena os pares (vizinho, peso) correspondentes às arestas incidentes ao vértice i . Para um grafo com V vértices e E arestas, a lista de adjacência ocupa espaço $O(V + E)$, enquanto uma matriz de adjacência ocuparia $O(V^2)$, independentemente da quantidade real de conexões. Como o grafo deste projeto

é esparso (15 arestas para 9 vértices, bem abaixo do máximo possível de $\binom{9}{2} = 36$), a lista de adjacência é significativamente mais eficiente em memória e também é a estrutura que permite acesso direto e rápido aos vizinhos de um vértice, operação central nos três algoritmos implementados.

A saída obtida ao exibir o grafo foi:

```
==== Grafo ====
0 : [(1, 4), (2, 2)]
1 : [(0, 4), (2, 1), (3, 5)]
2 : [(0, 2), (1, 1), (3, 8), (4, 10)]
3 : [(1, 5), (2, 8), (4, 2), (5, 6)]
4 : [(2, 10), (3, 2), (5, 3), (6, 7)]
5 : [(3, 6), (4, 3), (6, 1), (7, 9)]
6 : [(4, 7), (5, 1), (7, 4), (8, 6)]
7 : [(5, 9), (6, 4), (8, 2)]
8 : [(6, 6), (7, 2)]
```

Figura 1: Saída da representação do grafo

Observa-se que cada aresta aparece duas vezes na lista de adjacência (uma em cada direção), o que reflete corretamente a natureza não direcionada do grafo: a conexão entre as cidades u e v pode ser percorrida em ambos os sentidos com o mesmo custo. A classe **Grafo** implementa quatro responsabilidades principais:

- **Inicialização**: cria a lista de adjacência vazia, com uma sublista para cada vértice;
- **adicionar_aresta(origem, destino, peso)**: insere a aresta nos dois sentidos ($\text{origem} \rightarrow \text{destino}$ e $\text{destino} \rightarrow \text{origem}$), garantindo a propriedade de não-direcionalidade;
- **retorna_vizinho(origem)**: retorna a lista de pares (**vizinho**, **peso**) de um vértice, em tempo $O(1)$ (acesso direto por índice), operação fundamental usada por Dijkstra e Prim;
- **obter_arestas()**: percorre toda a lista de adjacência e retorna uma lista única de arestas no formato (**peso**, **u**, **v**), evitando duplicação ao considerar apenas pares em que $u < v$. Essa lista é a base para o algoritmo de Kruskal.

3 Parte 1: Caminhos Mínimos com Dijkstra

O algoritmo de Dijkstra resolve o problema do caminho mais curto a partir de uma única origem (*Single-Source Shortest Path*) em grafos com pesos não negativos. Sua estratégia é gulosa (*greedy*): a cada passo, o algoritmo seleciona, entre os vértices ainda não processados, aquele com a menor distância conhecida a partir da origem, e tenta melhorar (*relaxar*) as distâncias de seus vizinhos.

Estruturas mantidas pelo algoritmo

- **distancia[v]**: menor custo conhecido para chegar de **origem** até v (inicialmente ∞ para todos, exceto a origem, que recebe 0);

- **pais[v]**: vértice predecessor de v no caminho mínimo, usado posteriormente para reconstruir o caminho completo;
- **visitado[v]**: indica se v já teve sua distância definitivamente confirmada;
- **fila de prioridade (heapq)**: garante que o próximo vértice processado seja sempre o de menor distância acumulada, com custo $O(\log n)$ por inserção/remoção.

Operação de relaxamento

Para cada aresta $(u, v, peso)$, a operação de relaxamento verifica:

se $distancia[u] + peso < distancia[v] \implies distancia[v] \leftarrow distancia[u] + peso, \quad pais[v] \leftarrow u$

Essa verificação é repetida até que todos os vértices alcançáveis tenham sido processados, garantindo que **distancia[v]** contenha, ao final, o menor custo possível de **origem** até v .

Complexidade

Com fila de prioridade binária (**heapq**), a complexidade de tempo é $O((V + E) \log V)$, e a complexidade de espaço é $O(V + E)$ (armazenamento da lista de adjacência, dos vetores **distancia** e **pais**, e da fila de prioridade).

Reconstrução do caminho

A função **reconstruir_caminho(pais, destino)** percorre o vetor **pais** a partir do destino até a origem (onde **pais[origem] = None**), construindo o caminho em ordem inversa e depois invertendo-o para obter a sequência $origem \rightarrow \dots \rightarrow destino$.

3.1 Resultados obtidos

Tabela 2: Distâncias mínimas e predecessores a partir do vértice 0

Vértice	0	1	2	3	4	5	6	7	8
Distância	0	3	2	8	10	13	14	18	20
Predecessor (pai)	–	2	0	1	3	4	5	6	6

Caminho mínimo de 0 até 5:

$0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ Custo total = $2 + 1 + 5 + 2 + 3 = \mathbf{13}$

Caminho mínimo de 0 até 8:

$0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8$ Custo total = $2 + 1 + 5 + 2 + 3 + 1 + 6 = \mathbf{20}$

Ambos os resultados foram validados manualmente, percorrendo o vetor **pais** de trás para frente, confirmando a correção da implementação.

```
==== Dijkstra ====
Distancias: [0, 3, 2, 8, 10, 13, 14, 18, 20]
Pais: [None, 2, 0, 1, 3, 4, 5, 6, 6]
Caminho 0 -> 5: [0, 2, 1, 3, 4, 5]
Custo: 13
Caminho 0 -> 8: [0, 2, 1, 3, 4, 5, 6, 8]
Custo: 20
```

Figura 2: Saída Dijkstra

4 Parte 2: Árvore Geradora Mínima com Prim

O algoritmo de Prim constrói uma Árvore Geradora Mínima (*Minimum Spanning Tree* – *MST*): um subconjunto de arestas que conecta todos os vértices do grafo, sem formar ciclos, com o menor custo total possível.

Estratégia

1. Inicia-se com um único vértice (a árvore parcial contém apenas esse vértice);
2. A cada passo, escolhe-se, entre todas as arestas que conectam um vértice já incluído na árvore a um vértice ainda fora dela, a de menor peso;
3. Essa aresta é adicionada à árvore, e o novo vértice passa a fazer parte do conjunto "incluído";
4. Repete-se até que todos os V vértices estejam na árvore (a árvore final terá exatamente $V - 1$ arestas).

A implementação utiliza uma fila de prioridade (heapq) para armazenar candidatas (peso, u, v), sempre extraíndo a de menor peso. Arestas que apontam para vértices já visitados são descartadas (ignoradas com continue).

Complexidade

Assim como o Dijkstra, a complexidade de tempo é $O((V+E) \log V)$ com fila de prioridade binária, e a complexidade de espaço é $O(V + E)$.

4.1 Resultados obtidos

Saindo do vértice 0:

Tabela 3: Arestas selecionadas por Prim a partir do vértice 0

Origem	Destino	Peso
0	2	2
2	1	1
1	3	5
3	4	2
4	5	3
5	6	1
6	7	4
7	8	2

Custo total: $2 + 1 + 5 + 2 + 3 + 1 + 4 + 2 = \mathbf{20}$

Ordem de inclusão dos vértices: 2, 1, 3, 4, 5, 6, 7, 8 (o vértice 0 é o ponto de partida, já presente na árvore desde o início).

Saindo do vértice 5:

Tabela 4: Arestas selecionadas por Prim a partir do vértice 5

Origem	Destino	Peso
5	6	1
5	4	3
4	3	2
6	7	4
7	8	2
3	1	5
1	2	1
2	0	2

Custo total: $1 + 3 + 2 + 4 + 2 + 5 + 1 + 2 = \mathbf{20}$

Ordem de inclusão dos vértices: 6, 4, 3, 7, 8, 1, 2, 0.

```
==== Prim ====
Partindo do vértice 0:
Arestas escolhidas: [(0, 2, 2), (2, 1, 1), (1, 3, 5), (3, 4, 2), (4, 5, 3), (5, 6, 1), (6, 7, 4), (7, 8, 2)]
Custo total: 20
Ordem de inclusão: [2, 1, 3, 4, 5, 6, 7, 8]

Partindo do vértice 5:
Arestas escolhidas: [(5, 6, 1), (5, 4, 3), (4, 3, 2), (6, 7, 4), (7, 8, 2), (3, 1, 5), (1, 2, 1), (2, 0, 2)]
Custo total: 20
Ordem de inclusão: [6, 4, 3, 7, 8, 1, 2, 0]
```

Figura 3: Saída Prim

O custo total foi idêntico (20) em ambas as execuções, e o conjunto de arestas selecionadas também é o mesmo, apenas a orientação das tuplas e a ordem de inclusão mudaram,

pois ambas dependem do ponto de partida escolhido. Esse resultado é uma demonstração prática da propriedade do corte (cut property): o custo total de *qualquer* árvore geradora mínima de um grafo conexo é único, independentemente do vértice inicial utilizado pelo algoritmo de Prim. 6.

5 Parte 3: Árvore Geradora Mínima com Kruskal

O algoritmo de Kruskal também constrói uma Árvore Geradora Mínima, mas com uma estratégia diferente da de Prim: em vez de "crescer" a árvore a partir de um vértice, Kruskal trabalha diretamente sobre o conjunto global de arestas.

Estratégia

1. Todas as arestas do grafo são ordenadas em ordem crescente de peso;
2. Percorre-se a lista ordenada, e para cada aresta (u, v, peso) :
 - Se u e v ainda não estão no mesmo componente conexo, a aresta é adicionada à árvore, e os componentes de u e v são unidos;
 - Caso contrário, a aresta formaria um ciclo e é descartada.
3. O processo termina quando todas as arestas foram analisadas (ou quando a árvore já possui $V - 1$ arestas).

Estrutura Union-Find (Conjuntos Disjuntos)

Para verificar eficientemente se dois vértices pertencem ao mesmo componente, foi implementada a estrutura Union-Find, com duas operações:

- **find(x)**: retorna o representante (raiz) do conjunto ao qual x pertence, aplicando compressão de caminho (*path compression*), cada nó visitado durante a busca passa a apontar diretamente para a raiz, acelerando consultas futuras;
- **union(a, b)**: une os conjuntos de a e b , fazendo a raiz de um apontar para a raiz do outro.

Com compressão de caminho, cada operação tem complexidade amortizada próxima de $O(1)$ (mais precisamente $O(\log n)$ no pior caso sem *union by rank*).

Complexidade

A etapa dominante é a ordenação das arestas, com complexidade $O(E \log E)$. O processamento das arestas com Union-Find adiciona $O(E \log V)$. Complexidade total: $O(E \log E)$.

5.1 Resultados obtidos

Arestas ordenadas por peso (crescente):

$(1,2,1) - (5,6,1) - (0,2,2) - (3,4,2) - (7,8,2) - (4,5,3) - (0,1,4) - (6,7,4) - (1,3,5) - (3,5,6) -$
 $(6,8,6) - (4,6,7) - (2,3,8) - (5,7,9) - (2,4,10)$

(Cada tupla está no formato (peso, u, v), conforme retornado por `obter_arestas`.)

Tabela 5: Arestas selecionadas por Kruskal

Origem	Destino	Peso
1	2	1
5	6	1
0	2	2
3	4	2
7	8	2
4	5	3
6	7	4
1	3	5

Custo total: $1 + 1 + 2 + 2 + 2 + 3 + 4 + 5 = 20$

Tabela 6: Arestas descartadas por formarem ciclo

Origem	Destino	Peso
0	1	4
3	5	6
6	8	6
4	6	7
2	3	8
5	7	9
2	4	10

```

=== Kruskal ===
Arestas selecionadas: [(1, 2, 1), (5, 6, 1), (0, 2, 2), (3, 4, 2), (7, 8, 2), (4, 5, 3), (6, 7, 4), (1, 3, 5)]
Arestas descartadas: [(0, 1, 4), (3, 5, 6), (6, 8, 6), (4, 6, 7), (2, 3, 8), (5, 7, 9), (2, 4, 10)]
Custo total: 20
Arestas ordenadas por ordem crescente: [(1, 1, 2), (1, 5, 6), (2, 0, 2), (2, 3, 4), (2, 7, 8), (3, 4, 5), (4, 0, 1),
(4, 6, 7), (5, 1, 3), (6, 3, 5), (6, 6, 8), (7, 4, 6), (8, 2, 3), (9, 5, 7), (10, 2, 4)]

```

Figura 4: Saída Kruskal

6 Comparação entre os Algoritmos

6.1 Prim vs. Kruskal

Ambos os algoritmos resultaram em custo total idêntico (20) e, ao comparar os conjuntos de arestas (ignorando ordem e orientação), constatou-se que as arestas selecionadas são exatamente as mesmas:

$$\{(0, 2), (1, 2), (1, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8)\}$$

Esse resultado não é coincidência, mas também não é uma garantia absoluta de identidade de arestas em qualquer grafo. A teoria de grafos garante, através da propriedade

do corte (cut property), que o custo total de qualquer Árvore Geradora Mínima de um grafo conexo é único, ou seja, não existe uma MST "mais barata" que outra. Isso explica por que Prim e Kruskal sempre produzirão o mesmo custo total, independentemente do vértice inicial (no caso de Prim) ou da ordem de desempate (no caso de Kruskal).

Já a identidade exata das arestas só é garantida quando todos os pesos do grafo são distintos. Quando há pesos repetidos, podem existir múltiplas MSTs válidas, todas com o mesmo custo total, mas com conjuntos de arestas diferentes. No grafo deste projeto há um único empate de peso (peso 1, presente nas arestas (1, 2) e (5, 6)); ainda assim, como ambas as arestas de peso 1 foram necessárias para conectar componentes diferentes (nenhuma delas formava ciclo com a outra), esse empate não gerou ambiguidade, e os dois algoritmos convergiram para a mesma árvore.

6.2 Dijkstra vs. Prim/Kruskal: rota vs. rede

Observou-se que o custo do caminho mínimo de Dijkstra de 0 até 8 (20) é numericamente igual ao custo total da MST (20). Essa igualdade é uma coincidência numérica deste grafo específico, não uma propriedade geral.

Para evidenciar isso, basta comparar as arestas usadas em cada caso:

- Caminho de Dijkstra ($0 \rightarrow 8$): utiliza a aresta direta (6, 8) com peso 6 – uma aresta que não pertence à MST;
- MST (Prim/Kruskal): para conectar o vértice 8, utiliza as arestas (6, 7) e (7, 8), com pesos $4 + 2 = 6$, o mesmo custo numérico, mas alcançado por um caminho diferente, com uma aresta extra.

Isso ilustra exatamente a diferença conceitual central do projeto:

- Dijkstra responde: "qual o menor custo para ir especificamente de 0 até 8?", um problema de rota.
- Prim/Kruskal respondem: "qual o menor custo para conectar todos os 9 vértices em uma única rede?", um problema de conectividade global.

Se o peso da aresta (6, 8) fosse alterado de 6 para, por exemplo, 100:

- O custo do caminho de Dijkstra $0 \rightarrow 8$ mudaria, pois o algoritmo simplesmente deixaria de usar essa aresta cara e recalcularia a melhor rota (provavelmente passando por $6 \rightarrow 7 \rightarrow 8$, com custo $14 + 4 + 2 = 20$, mantendo o valor 20 – ou outra rota, dependendo dos demais pesos);
- O custo da MST poderia mudar ou não, dependendo se essa aresta era ou não utilizada na árvore e, neste grafo específico, ela não é utilizada pela MST, então o custo da MST permaneceria 20.

Esse exercício mental confirma que os dois valores são conceitos independentes, que neste grafo particular convergiram para o mesmo número por coincidência.

7 Respostas às Perguntas de Análise

7.1 Parte 1: Dijkstra

1 Por que BFS não seria suficiente para resolver esse problema?

O BFS (Busca em Largura) encontra o caminho com o menor número de arestas entre dois vértices, assumindo implicitamente que todas as arestas têm o mesmo "custo" (peso 1). Em um grafo ponderado, o caminho com menos arestas não é necessariamente o caminho de menor custo total. O BFS processa os vértices em ordem de distância em número de saltos, e não em ordem de custo acumulado, ele não possui mecanismo para comparar e priorizar caminhos com base em pesos diferentes. Por isso, é necessário um algoritmo como o Dijkstra, que utiliza uma fila de prioridade ordenada pelo custo acumulado real.

2 O que significa relaxar uma aresta?

"Relaxar" a aresta $(u, v, peso)$ significa verificar se é possível melhorar (diminuir) a melhor distância conhecida até v , passando por u . Formalmente, a condição de relaxamento é:

$$\text{se } distancia[u] + peso < distancia[v]$$

Se a condição for verdadeira, atualiza-se $distancia[v] \leftarrow distancia[u] + peso$ e registra-se u como o novo predecessor de v ($pais[v] = u$). O termo "relaxar" reflete a ideia de afrouxar uma restrição anterior (a estimativa antiga de distância), substituindo-a por uma estimativa melhor.

3 Qual a diferença entre encontrar o caminho com menos arestas e o caminho de menor custo?

São critérios de otimização distintos. O caminho com menos arestas minimiza a quantidade de saltos entre origem e destino, independentemente dos pesos. O caminho de menor custo minimiza a soma dos pesos das arestas percorridas, podendo, para isso, utilizar mais arestas.

No grafo deste projeto, por exemplo, o caminho $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8$ usa apenas 6 arestas, mas tem custo $4 + 5 + 2 + 3 + 1 + 6 = 21$. Já o caminho encontrado pelo Dijkstra, $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8$, usa 7 arestas, porém tem custo $2 + 1 + 5 + 2 + 3 + 1 + 6 = 20$, menor, apesar de ter mais arestas. Isso evidencia que "mais curto" (em arestas) e "mais barato" (em custo) são propriedades independentes.

4 O algoritmo de Dijkstra funcionaria se houvesse pesos negativos? Justifique.

Não. O Dijkstra baseia-se na premissa de que, uma vez que um vértice é marcado como visitado (ou seja, sua distância é considerada definitiva), nenhuma rota futura poderá produzir um caminho ainda mais curto até ele. Essa premissa só é válida se todos os pesos forem não negativos, pois adicionar arestas a um caminho nunca pode diminuir seu custo total.

Com pesos negativos, é possível que um vértice já "finalizado" seja alcançado posteriormente por um caminho que passa por uma aresta de peso negativo, resultando

em um custo total menor do que o já registrado, mas o algoritmo, por já ter marcado esse vértice como visitado, nunca reconsiderará essa possibilidade, produzindo um resultado incorreto. Para grafos com pesos negativos (sem ciclos negativos), o algoritmo adequado é o Bellman-Ford, que permite múltiplas relaxações de cada aresta.

7.2 Parte 2: Prim

1 Qual é a diferença entre o objetivo de Prim e o objetivo de Dijkstra?

Embora ambos sejam algoritmos gulosos que utilizam fila de prioridade e compartilhem semelhanças estruturais, seus objetivos são diferentes:

- Dijkstra minimiza o custo do caminho de uma origem fixa até cada vértice individualmente produzindo, no final, uma árvore de caminhos mínimos (*shortest path tree*), onde a soma dos pesos no caminho até cada vértice é a menor possível;
- Prim minimiza o custo total de uma árvore que conecta todos os vértices entre si, sem se preocupar com a distância individual da origem até cada um, o objetivo é a conectividade global de menor custo, não rotas específicas.

Em resumo: Dijkstra otimiza distâncias individuais a partir de um ponto; Prim otimiza o custo agregado da rede como um todo.

2 Por que a árvore geradora mínima não pode conter ciclos?

Por definição, uma árvore é um grafo conexo e acíclico. Uma árvore geradora com V vértices possui exatamente $V - 1$ arestas, o número mínimo necessário para conectar todos os vértices.

Se uma aresta adicional fosse incluída formando um ciclo, ela seria, por definição, redundante: os dois vértices que ela conecta já estariam unidos por um caminho alternativo dentro da árvore. Incluir essa aresta apenas aumentaria o custo total sem contribuir para a conectividade, violando o objetivo de minimização. Por isso, o algoritmo de Prim só adiciona arestas que conectam um vértice novo (ainda fora da árvore) a um vértice já incluído, garantindo que nenhum ciclo seja formado.

3 O resultado de Prim depende do vértice inicial? Explique.

O custo total da árvore gerada não depende do vértice inicial, essa é uma consequência da propriedade do corte (*cut property*), que garante que toda MST de um grafo conexo possui o mesmo peso total, independentemente de como ela é construída.

Isso foi demonstrado experimentalmente neste projeto: executando Prim a partir do vértice 0 e a partir do vértice 5, obteve-se em ambos os casos o mesmo custo total (20) e o mesmo conjunto de arestas, apenas a ordem de inclusão dos vértices e a orientação das arestas na saída foram diferentes, refletindo apenas o caminho de exploração do algoritmo, e não uma diferença real na solução encontrada.

Em grafos com pesos repetidos (empates), o vértice inicial poderia influenciar qual MST específica (entre várias possíveis, todas de mesmo custo) é encontrada mas o custo total continuaria sendo o mesmo em qualquer caso.

7.3 Parte 3: Kruskal

1 Como Kruskal detecta se uma aresta formaria ciclo?

Kruskal utiliza a estrutura de conjuntos disjuntos (Union-Find). Cada vértice começa em seu próprio conjunto. Para cada aresta $(u, v, peso)$ analisada (em ordem crescente de peso), o algoritmo chama $find(u)$ e $find(v)$ para obter os representantes (raízes) dos conjuntos aos quais u e v pertencem:

- Se $find(u) == find(v)$, ambos já pertencem ao mesmo componente conexo ou seja, já existe um caminho entre eles dentro da árvore parcial. Adicionar essa aresta criaria um ciclo, então ela é descartada;
- Se $find(u) != find(v)$, os vértices estão em componentes diferentes; a aresta é adicionada à MST, e os dois conjuntos são unidos via $union(u, v)$.

2 Por que ordenar as arestas é uma etapa essencial?

O algoritmo de Kruskal é guloso: ele constrói a MST tomando, a cada passo, a decisão localmente ótima de incluir a menor aresta disponível que não forma ciclo. Essa estratégia só produz uma solução globalmente ótima (custo mínimo) se as arestas forem consideradas em ordem crescente de peso, garantindo que, sempre que uma conexão entre dois componentes for necessária, a opção mais barata disponível naquele momento seja escolhida primeiro.

Sem a ordenação, o algoritmo poderia incluir arestas mais caras antes de arestas mais baratas que conectariam os mesmos componentes, resultando em uma árvore de custo total maior que o mínimo possível.

3 Prim e Kruskal devem produzir sempre o mesmo custo total? Justifique.

Sim, sempre. Essa garantia decorre da propriedade do corte (cut property): para qualquer partição do conjunto de vértices em dois grupos não vazios, a aresta de menor peso que cruza essa partição (o "corte") pertence a alguma MST do grafo. Como tanto Prim quanto Kruskal, em cada passo, sempre escolhem a aresta de menor peso que conecta componentes diferentes (Prim de forma "local", a partir da árvore parcial; Kruskal de forma "global", percorrendo a lista ordenada), ambos estão obedecendo à mesma propriedade fundamental, e portanto convergem para árvores de mesmo custo total, mesmo que, em grafos com pesos repetidos, o conjunto exato de arestas possa diferir entre múltiplas MSTs igualmente válidas.

No grafo deste projeto, ambos os algoritmos produziram custo total **20**, com o mesmo conjunto de arestas, confirmando a teoria na prática.

8 Conclusão

A implementação dos três algoritmos permitiu compreender, na prática, as diferenças fundamentais entre problemas de roteamento (Dijkstra) e problemas de conectividade de rede de menor custo (Prim e Kruskal). Embora compartilhem semelhanças estruturais, estratégias gulosas, uso de filas de prioridade e operação sobre a mesma representação de grafo, cada algoritmo responde a uma pergunta diferente sobre o mesmo conjunto de dados.

Os resultados experimentais confirmaram as garantias teóricas estudadas: a unicidade do custo da MST (independente do algoritmo e do vértice inicial), a importância da ordenação no Kruskal, a impossibilidade de relaxamento correto com pesos negativos em Dijkstra, e a distinção entre minimizar *arestas* versus minimizar *custo*.

Do ponto de vista de boas práticas de desenvolvimento, o projeto foi estruturado de forma modular com a representação do grafo, cada algoritmo e o ponto de entrada (`main.py`) em arquivos separados o que facilitou a depuração incremental e a validação independente de cada componente antes da integração final.